

# Script-Controlled ACL Lab – Phase 8

## Milestone 7: Creation of Access Control Lists (Delete ACL)

To round off our complete record security framework lifecycle, we must implement a strict table-level deletion boundary rule. This ensures structural data removal actions are locked away securely, allowing only specialized security-cleared profiles to eliminate entries.

| Form Parameters | Target Settings / Value Mapping               |
|-----------------|-----------------------------------------------|
| Type            | record                                        |
| Operation       | delete                                        |
| Name            | Institution Details [u_institution_details]   |
| Active          | true (Checked)                                |
| Requires Role   | bb4 (Custom Infrastructure Role)              |
| Data Conditions | None (Leave empty to apply to table globally) |

### Step-by-Step Implementation Workflow:

1. In the Application Navigator side-bar menu, search for **System Security** → **Access Control (ACL)**.
2. Ensure your admin session remains elevated with the **security\_admin** flag checked active via your profile menu.
3. Click the blue **New** button positioned atop the data list view layout grid.
4. Fill out the operational block parameter fields precisely using the table framework grid mapped above.
5. Locate the **Requires role** embedded related list matrix at the bottom segment of the core workspace layout.
6. Double-click *Insert a new row...*, type or lookup your custom **bb4** access tracking role, and save it.
7. Click the main header action **Submit** button to update system schema protections.

## Milestone 8: Full Security Framework Verification Matrix

To guarantee that cumulative rule inheritance scales securely without collisions, execute the impersonation validation loops outlined below:

| Assigned Roles Combo           | Expected UI Behavior Matrix Boundaries                                                                                            |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| bb1 Only                       | Can strictly <b>Read</b> records containing <b>Branch = EEE</b> . New button hidden. Edit/Delete operations are blocked entirely. |
| bb1 & bb2 Combined             | Can <b>Read</b> EEE records and gains active visibility to initialize records via the <b>New Button</b> .                         |
| bb1 & bb2 & bb3 Combined       | Can <b>Read</b> , launch form creations, and actively perform <b>Write / Field Modifications</b> across EEE records.              |
| bb1 & bb2 & bb3 & bb4 Full Set | Gains complete Lifecycle Rights: can <b>See, Create, Edit, along with Delete the EEE branch records</b> successfully.             |
| Admin User                     | Bypasses context boundaries completely. Holds full administrative access across all branch types (ECE, CSE, EEE).                 |
| Standard/No Role User          | Strictly blocked by context filters. No record visibility or UI actions are permitted.                                            |

## Laboratory Wrap-up & Comprehensive Outcome

**Outcome Summary:** By completing this rigorous hands-on laboratory environment, learners gain a comprehensive understanding of how script-controlled READ, WRITE, CREATE, and DELETE Access Control Lists (ACLs) enforce strict record-level contextual security across a schema. You have observed firsthand how user roles and live record field criteria evaluate side-by-side to construct structural data access choices. This layer ensures that sensitive functional information remains safely protected from unauthorized viewing, malicious or accidental modification, creation, and deletion anomalies across native forms, background lists, and external Service Portal interfaces.